VCD Assignment: Notes

Author: Diego Minguzzi Date: 09/03/2025

The developed Python program is designed to generate a Verification Control Document (VCD) report from requirement and test report data. The tool is structured to be reusable and extendable, with separation of concerns into multiple modules.

Design

- **Entities**: Implements core domain classes (e.g., requirements, test reports, specifications).
- **Service**: Handles the asynchronous loading and parsing of data
- **Storage**: Provides collections to manage and store entities, with the logic to create relations between entitites
- **VCD**: Implements the report generation logic, using the Builder-Director pattern to allow extensions for additional output formats.

Data validation

The program is designed to handle invalid data gracefully, skipping any problematic entities while raising warnings. The tool attempts to continue processing without halting unless an unrecoverable error occurs. On the other hand, the tool does not check whether requirements are actually structured into three levels, or if a specification is linked to requirements at the same hierarchical level. The tool makes the hypothesis that the relations between data entities are correct, it just validates the entities.

Logging and output

By default, the tool suppresses output to the console, unless the log level is set to DEBUG or INFO by command line, or if warnings are raised during the data loading phase.

Asynchronous Programming

The program leverages asynchronous I/O operations to speed up data loading and report generation. However, certain phases (e.g., entity validation and storage) are processed in a single thread due to the Global Interpreter Lock (GIL) limitation in Python. Once all data are loaded, it is possible to run in parallel the report generation phase: each table can be generated in parallel with the others. The tool contains the code to do so, using the –parallel command line argument. With the current dataset, the overhead in creating the separate processes and sending input data to each process, exceeds the speedup given by the multiple processes. The speedup was actually measured and tested by inserting a sleep in the code: the sleep was removed in the delivered version.

Unit Testing

A basic unit test is provided for validating the retrieval of descendant requirements. Full unit test coverage for all classes is not included, though additional tests would be necessary for a production-grade tool.

Environment

The tool is developed using Python 3.10.12 and tested on Linux Lubuntu 22. The root directory contains a `README.md` file with instructions for setting up a separate environment and installing dependencies via `pip`.

External Libraries

The program relies on the following Python libraries:

- aiofiles: For asynchronous file handling (link).
- pyyaml: for parsing YAML files to process test reports (link)
- tabulate: for formatting and generating markdown tables for the final VCD report (link).